ISJ107V

Assignment 4

Functional Prototype and Presentation

IntelliClass

AI-Assisted Academic Management Platform

Name

Alicia Tau

Student number

223230660

31 October 2026

Emerging technology: Artificial Intelligence, used only as an assistant.

Question 1 — Prototype readiness

This section defines what the prototype demonstrates and how an assessor can run it.

1.1 Title, problem, users, purpose and technology

Project title. IntelliClass — AI-Assisted Academic Management Platform.

Problem addressed. Schools struggle to see whether academic work is done, teachers spend a long time preparing assessments from study notes, and parents often hear about progress only at report time. There is no single place where the teacher, the learner and the parent can look at the same record.

Intended users. The prototype supports four roles. An administrator manages users, classes and subjects. A teacher uploads study material, creates quizzes and reviews anything the AI drafts. A learner opens materials, completes a published quiz and can ask why an answer was wrong. A parent views quiz results and notes from the teacher.

Prototype purpose. The prototype shows one complete path: a teacher turns an uploaded PDF into a reviewed quiz, a learner answers it, the score is stored, and a wrong answer can be explained from the same notes. The aim is to show learner accountability, less preparation time for the teacher, and a clearer view for the parent.

Emerging technology. Artificial intelligence, through a pre-trained model on Hugging Face. The model is called only from the server. It drafts questions and explains a wrong answer. It does not decide who passes. If the service is unavailable, the server builds a simpler draft from the notes so the demonstration can continue.

1.2 Requirements and feature status

The five requirements that the live demonstration is built around are FR-01 to FR-05. FR-06 and FR-07 are also available. FR-08 and FR-09 exist in the code but are not part of the demonstration screens.

Table 1 Functional requirements demonstrated by the prototype

ID	Requirement	Status
FR-01	A user signs in and opens only the dashboard for that role.	Implemented
FR-02	An administrator manages users, classes and subjects.	Implemented
FR-03	A teacher uploads a PDF and the system extracts its text.	Implemented
FR-04	AI drafts a quiz from that text. The teacher reviews it before publish.	Implemented
FR-05	A learner completes a published quiz and the score is stored.	Implemented
FR-06	A wrong answer can be explained from the question and the notes.	Implemented
FR-07	A parent can see results, and a teacher can write to a parent.	Implemented
FR-08	Homework can be assigned, submitted and marked.	Partial
FR-09	A term report can be prepared from stored results.	Partial
FR-10	A phone app, scanned-PDF reading, or an AI pass mark.	Deferred

Partial means the database and server routes exist, but the demonstration screens hide homework and term reports so the assessed path stays on materials, quizzes and explanations. Deferred means those items are outside this prototype on purpose.

1.3 Setup and access

Software. Node.js and npm, PostgreSQL, and a current browser (Chrome, Edge or Firefox). No special AI hardware is required.

Hardware. A laptop with about 8 GB of memory, space for Node modules and PostgreSQL, and an internet connection when the live Hugging Face model is used.

- From the project folder, run npm install.
- Copy server/.env.example to server/.env and client/.env.example to client/.env.
- Set DATABASE_URL to the local PostgreSQL database. The server applies server/db/schema.sql on startup.
- Set HUGGINGFACE_TOKEN for the live model. Without it, quiz generation uses the local fallback.
- Start the API with npm run dev:server (http://localhost:4000).
- Start the interface with npm run dev:client (http://localhost:5173).

Demonstration sign-in. The first time the server starts with an empty user table it creates an administrator: admin@int.com with password Admin@123. Accounts created later for a teacher, learner or parent receive the temporary password Password@1. These are local demonstration credentials only.

Sample data. Create one class, one subject, one teacher, one learner in that class, and one parent linked to that learner. Upload a text-based PDF, not a scan. A short set of notes with a glossary gives the AI, and the fallback, enough material for a draft.

Repository. The working copy is the IntelliClass project used for the earlier assignments in this module. Record the submitted Git or Brightspace location here if a link is required: _____.

Question 2 — Working prototype

The prototype demonstrates the central value of IntelliClass: study notes become a reviewed quiz, the learner's result is kept, and a wrong answer can be explained.

2.1 End-to-end workflow

The demonstrated workflow has navigation, checks, a visible result and a clear error when the input is not usable.

- The administrator signs in, creates the class, subject and users, and links the learner and the teacher.
- The teacher signs in, opens Study materials, and uploads a PDF. A file that is not a PDF is rejected. A PDF with no readable text is rejected with a message that asks for a text-based file.
- The teacher opens Quizzes, chooses the material and chapters, sets a difficulty mix, and asks for a draft.
- The draft is shown for editing. It stays unpublished until the teacher approves it. Rejected or unfinished questions are not offered to learners.
- The learner signs in, opens the published quiz, submits answers, and sees the stored score.

- If an answer is wrong, the learner can ask for an explanation. The parent can open the same learner's quiz results.

Validation covers a missing login, a short password on account creation (at least 8 characters), the wrong role on a protected action, a non-PDF upload, and a quiz submitted before it is published.

2.2 Emerging technology in the workflow

The AI step sits in the middle of the workflow, not beside it. After the PDF text is stored, the server sends the selected chapters and a fixed instruction to Hugging Face: write questions only from those notes, in the requested difficulty mix. The reply is parsed as structured questions. Each question is checked for four distinct options, a correct answer that matches one option, and wording that is not an empty or banned choice. Accepted questions are saved as a draft and shown to the teacher. That draft is the observable result.

A second AI result appears after the learner submits. For an incorrect answer, the server sends the question, the learner's choice, the correct answer and the relevant notes. The explanation is returned on the quiz screen. The model never receives the whole learner record, and it never sets a pass or fail.

If Hugging Face fails or returns too little, the same request continues with a local builder that writes questions from definitions in the notes. The teacher still reviews that draft. On 31 October 2026 a glossary sample of eight terms produced 8 questions, and all 8 passed the same validator, in about 2 milliseconds. A short note with no glossary produced 3 weak questions and the validator accepted none of them, which is why the screen tells the teacher to choose more of the notes.

2.3 Persistent data

PostgreSQL stores users, classes, subjects, the links between them, the PDF record and its extracted text, the quiz, each question, and the learner's submission and score. Restarting the server does not remove a published quiz or a stored result. The schema is applied from server/db/schema.sql when the API starts.

2.4 Security and reliability

- Passwords are stored as bcrypt hashes. The temporary password is not kept in the database as plain text.
- Sign-in returns a JWT. Protected routes use that token, and requireRole blocks a learner from publishing a quiz or calling the teacher AI route.
- The Hugging Face token stays in server/.env. The browser never receives it.
- Helmet and CORS limit the API to the configured client origin. A bad request returns a short message and an HTTP status, not a stack trace.
- A learner only receives published quizzes for a linked class. A parent only sees a linked child.

Table 2 Prototype evidence for the assessed workflow

ID	Status	Feature	Test	Evidence
FR-01	Completed	Role dashboard	TC-01	Live sign-in
FR-03	Completed	PDF text extraction	TC-02	Upload error or stored text
FR-04	Completed	AI draft, then review	TC-05	Draft on screen
FR-05	Completed	Score stored	TC-04	Result after submit

ID	Status	Feature	Test	Evidence
FR-06	Completed	Answer explanation	TC-06	Explanation text
ETR-01	Completed	Hugging Face or fallback	TC-05	8 of 8 local questions valid

Question 3 — UML of the prototype

The diagrams describe the prototype that will be demonstrated, not every screen that was designed earlier.

3.1 Use-case diagram

Administrator, teacher, learner and parent are the people in the demonstration. The AI service is a supporting system. Sign-in is shared. Only a teacher generates and publishes a quiz. Only a learner asks for an explanation. A parent views progress.

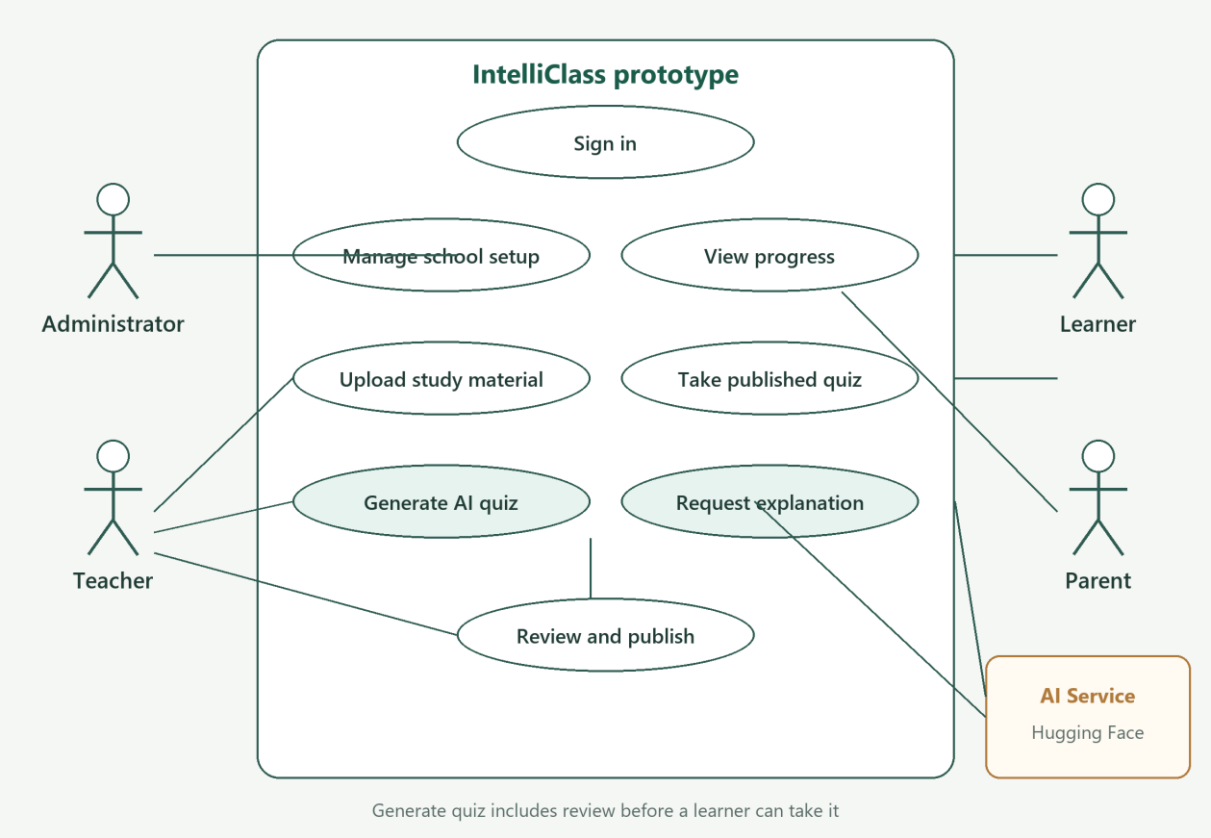


Figure 1 Use cases included in the demonstration

3.2 Activity diagram

The activity is the quiz path. A PDF without readable text stops with an error. If the model does not return usable questions, the server builds a draft from the notes and still sends it for review. Edit or reject does not publish the quiz. A wrong answer is the point where the explanation step runs.

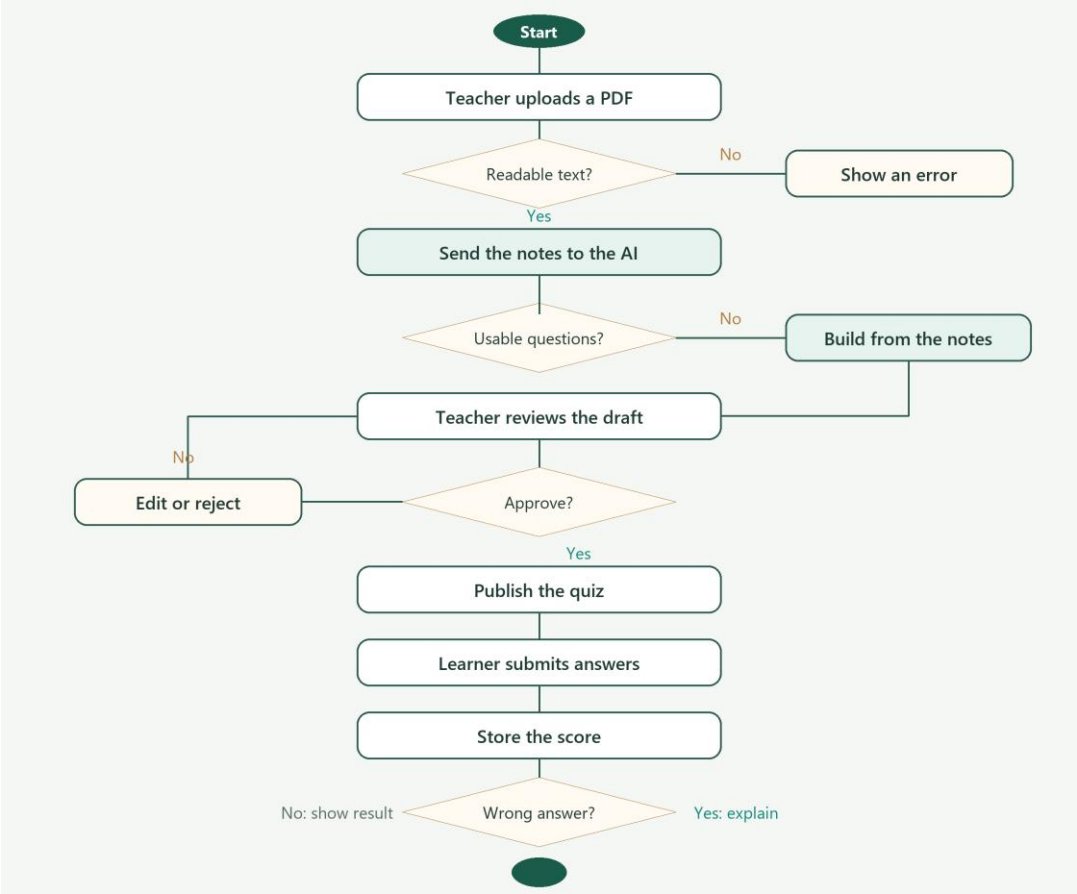


Figure 2 Activity diagram for AI-assisted quiz generation and assessment

3.3 Class diagram

The main records are the user, the class, the subject, the study material, the quiz, its questions and the stored result. Role is an attribute of User. AiService is external. A parent is linked to a learner rather than owning the result directly.

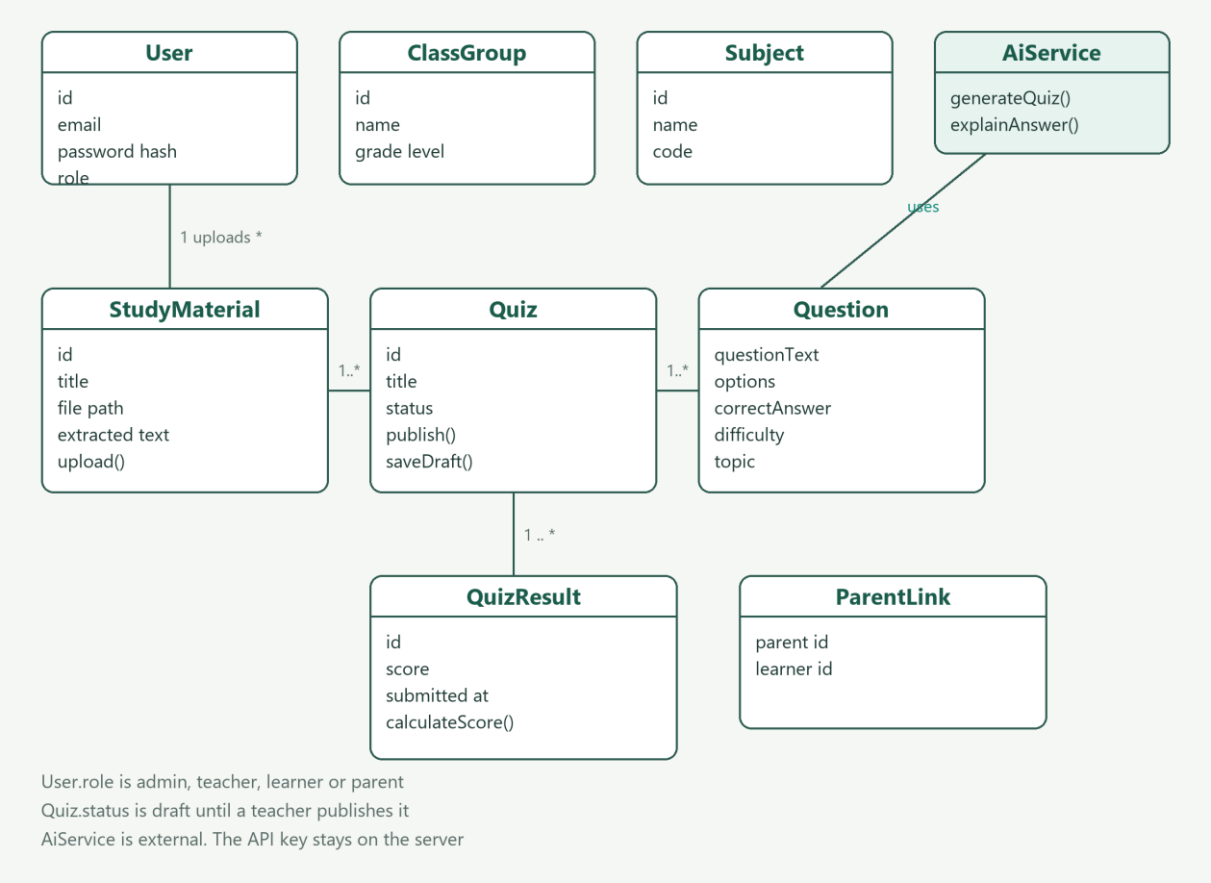


Figure 3 Main classes in the prototype

3.4 Sequence diagram

Time runs from the teacher, through the React screen and the Express API, to PostgreSQL and Hugging Face. The model is called only after the text has been saved, and the draft is validated before it is stored.

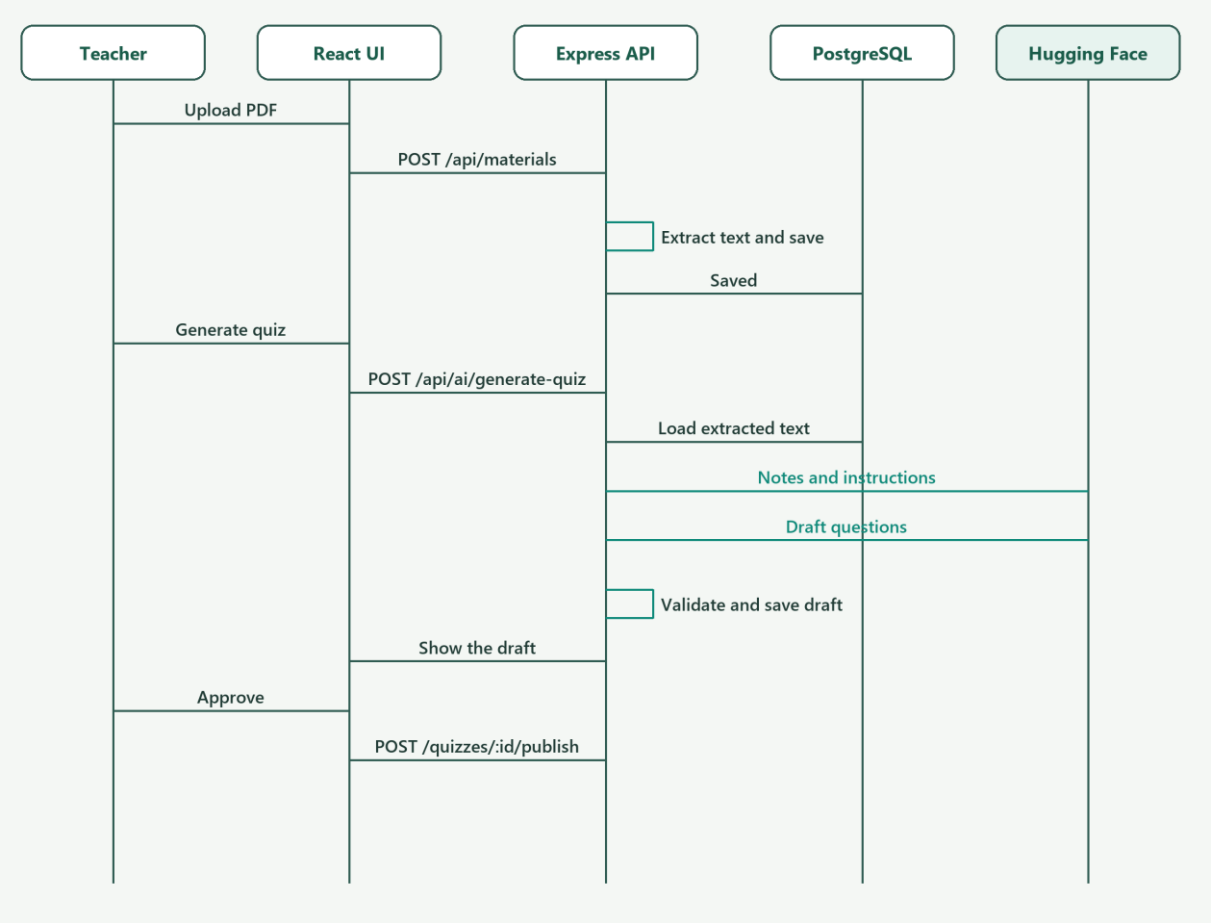


Figure 4 Sequence for generating and publishing an AI quiz

3.5 Deployment and differences from the design

The browser holds the React interface. The Node server holds the rules, the file upload and the AI call. PostgreSQL holds the records. Hugging Face is outside the school network. The API key never leaves the server.

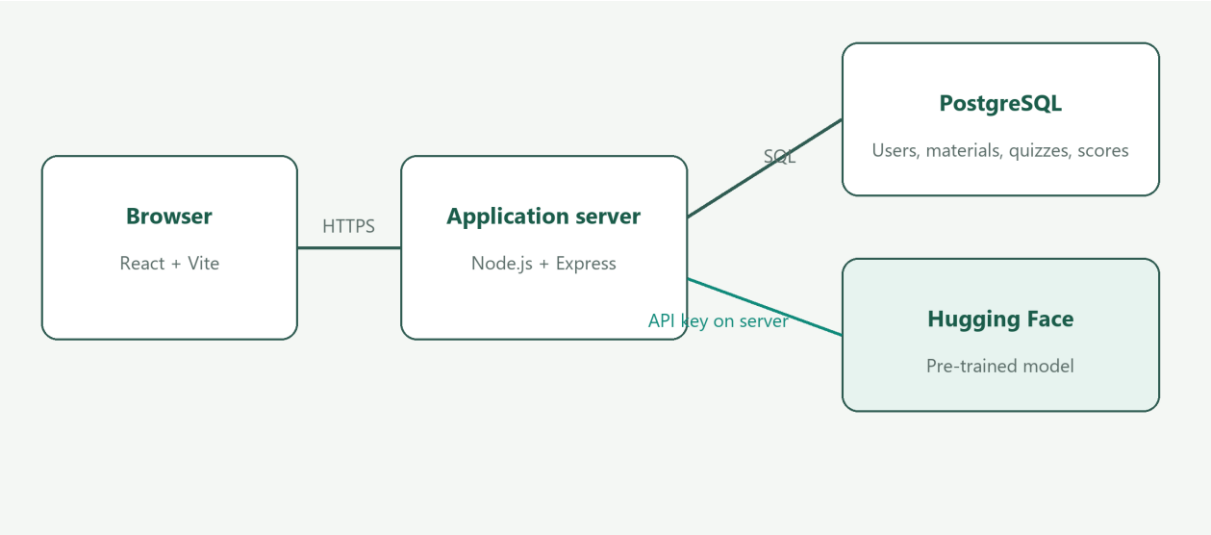


Figure 5 Deployment of the prototype

Three differences matter between the earlier design and this build.

- The design drew administrator, teacher, learner and parent as subclasses. The database stores one users table and a role value. The class diagram follows that simpler structure.

- Homework and term reports were in the original scope and are still in the schema and routes. Their menus are switched off for this demonstration so the assessed path is the quiz workflow.
- The sequence shows Hugging Face. When that call fails, the same API step uses a local builder instead of stopping with an empty quiz. The teacher still has to approve the result.

Question 4 — Evidence, measures and limits

The tests below cover the main path, a bad input, access control, stored data and the AI component.

4.1 Test cases

Table 3 Representative tests

ID	Case	Expected	Actual	Status
TC-01	Sign in with a wrong password	Access is refused	The login route returns 401 and does not open a dashboard	Pass
TC-02	Upload a file that is not a PDF	The upload is rejected	The materials route returns 400 before a record is stored	Pass
TC-03	A learner calls quiz generation	The action is refused	requireRole allows only a teacher or administrator	Pass
TC-04	Submit a quiz that is still a draft	No score is stored	The submit route rejects anything that is not published	Pass
TC-05	Draft 8 questions from glossary notes	Usable questions from the notes	Executed 31 Oct 2026: 8 built and 8 accepted by the validator	Pass
TC-06	Draft from a short note with no glossary	A weak draft is not silently accepted	Executed 31 Oct 2026: 3 built and 0 accepted, because they lacked four options	Pass

TC-01 to TC-04 follow the responses coded in the login, upload, role and submit routes, and they are the cases to repeat in the live demonstration. TC-05 and TC-06 were executed against the prototype's own generator and validator, without calling Hugging Face, so the fallback can be shown even if the external service is down.

4.2 Quantitative measures

Measure 1 — fallback completeness. On a note with eight defined terms, the local builder produced 8 of the 8 requested questions, and the validator accepted 8 of 8 (100% of that sample). The five timed runs averaged about 2 milliseconds, so the fallback does not delay the demonstration.

Measure 2 — fallback rejection. On a short paragraph with too few definitions, 0 of 3 built questions were accepted. The check is doing useful work: it refuses a draft that does not meet the four-option rule instead of publishing a broken quiz. The live model is judged the same way. A draft is kept only after that check, and only a teacher can publish it.

4.3 Limitations and corrective actions

Limitation 1. The model, and the simpler fallback, can still miss the point of the notes or need a glossary before they produce a full set of questions. A scanned PDF has no text, so generation cannot start.

Corrective action. Keep the teacher review step in the final system. Add a clearer message when a PDF has no text, and add optical character recognition only for those scans. Continue to reject questions that fail the validator.

Limitation 2. Homework marking and term reports are built but hidden, and the explanation depends on an external service that can be slow or unavailable.

Corrective action. Turn those screens back on after the demonstration checklist is signed off. Keep the local fallback, cache a successful explanation against the question, and never let the model set the learner's final mark.

Question 5 — Presentation and live demonstration

The presentation is 5 minutes. The live demonstration is 10 minutes. The slides are in IntelliClass_Presentation.pptx, in the Assignment 2 folder of this project, and they follow the order required here.

5.1 Five-minute structure

- Problem — missed homework, slow quiz preparation, parents who hear late, and no single record.
- Proposed solution — one platform for the administrator, teacher, learner and parent.
- Architecture — React, Node and Express, PostgreSQL, and Hugging Face called only from the server.
- Emerging technology — draft a quiz, summarise stored scores, explain a wrong answer. The teacher approves first.
- Prototype features — sign-in, PDF text, chapter and difficulty choice, publish, stored score, explanation, parent view.
- Results and limitations — the notes check and the teacher gate hold; vague output, scanned PDFs and a failed model call are the limits.

Speaker notes with a rough time for each slide are in IntelliClass_Presenter_Notes.docx beside the slides.

5.2 Ten-minute demonstration and fallback

- Minutes 1–2. Sign in as the administrator and show the class, subject and linked users. Sign out.
- Minutes 2–5. Sign in as the teacher. Upload the prepared PDF. Show the extracted text, generate a draft, edit one question, and publish.
- Minutes 5–8. Sign in as the learner. Complete the quiz, show the stored score, and request an explanation for one wrong answer.
- Minutes 8–10. Sign in as the parent and open that learner's result. If time remains, show that a learner cannot open the generate action.

Fallback. Prepare the PDF and the user links before the session. If Hugging Face does not respond, generate again: the server fills the draft from the notes and the review step is unchanged. If the network fails entirely, use a quiz that was published earlier and is already stored in PostgreSQL, then demonstrate the learner submission and the explanation request. Do not type a new password on screen; use the demonstration accounts already created.

5.3 Slides and timing

The deck has nine slides, in the green and paper colours of the running application. Each slide has a heading and a short explanation, not a paragraph. The five-minute talk stays on those slides. The ten-minute demonstration is done in the browser, not by reading the slides again.

5.4 Questions an assessor may ask

- Why is the model called from Express rather than from React? So the API key and the learner record stay on the server.
- Why can a learner not see a generated quiz immediately? Because status remains draft until the teacher publishes it.
- What happens when the model is wrong? The validator drops a bad question, and the teacher can edit or reject the rest. The model does not set the mark.
- Where is the score kept? In PostgreSQL, on the quiz submission, after the learner submits a published quiz.

References

Express. (2026). Express routing. https://expressjs.com/

Hugging Face. (2026). Inference providers. https://huggingface.co/docs/inference-providers

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. https://www.postgresql.org/docs/

React. (2026). React reference. https://react.dev/

Jones, M., Bradley, J. and Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. https://www.rfc-editor.org/rfc/rfc7519

Declaration

I declare that IntelliClass is my project for ISJ107V and that this report describes the prototype I will demonstrate. An AI assistant in Cursor was used to organise this report and to draw the figures from the implemented program. I have checked the description against the system. The source code, the design decisions and the live demonstration are my own work.

Signature _____ Date _____